Splunk Investigation Queries

Sysmon lab queries and password reset / jump box detection

Read this first

- None of the Part B queries were run against a real Splunk environment while writing this. Treat them as drafts. Before relying on one, create the scenario with test accounts and confirm it returns the expected rows, then confirm it returns nothing on a normal day.
- Field names (Subject_Account_Name, Target_Account_Name, Logon_Type and so on) are from the standard Splunk Add-on for Microsoft Windows. If yours differ, run B2 and edit the queries to match.
- Splunk dates in earliest and latest are month/day/year. 10/11/2026 means 11 October 2026.
- Replace index=* with your real index once you know it. Wildcard index searches are slow.
- To turn B10 into an alert: use earliest=-4h in place of -24h, schedule it every 15 minutes, and trigger when the number of results is greater than 0.
- A hit shows the sequence happened. It does not prove malicious intent. One person with several accounts can do exactly this legitimately, so use hits to decide what to review.
- Copy queries from the .md file or from GitHub, not from this PDF. Copying from a PDF can break line endings and quote marks.

Contents

A1 Trojan started cmd.exe (when?)

A2 New user added with net user /add

A3 Script file dropped on disk (.ps1)

A4 Firewall rule added

A5 Process chain (parent and child) by ProcessGuid

A6 Network connections made by one process

B1 Check the data exists

B2 Check the field names

B3 How many users changed their own password (exact name match)

B4 Same, treating firstname_xxx accounts as one person

B5 Discover which account name pairs exist

B6 Entra ID (Azure AD) self-service password reset

B7 Okta password reset or update

B8 Raw timeline for one user on one day (abc*)

B9 Prove the password change came from the jump box (domain controller side)

B10 MAIN DETECTION: RDP logon, then password change/reset, then target account logs in

B11 One line per incident (replace the last two lines of B10)

Part A - Sysmon investigation (lab file sysmon.json)

A1. Trojan started cmd.exe (when?)

Finds the process-create event (EventID 1) where cmd.exe was started, with its parent. Replace YOUR-HOST with the host name.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=1 "Event.EventData.Image"="*\\cmd.exe"
| sort + _time
| table _time Event.EventData.ParentImage Event.EventData.Image Event.EventData.CommandLine
```

A2. New user added with net user /add

Shows net.exe / net1.exe process creations, including the username and password typed on the command line.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=1
("Event.EventData.Image"="*\\net.exe" OR "Event.EventData.Image"="*\\net1.exe")
| sort + _time
| table _time Event.EventData.Image Event.EventData.CommandLine Event.EventData.ParentImage
```

A3. Script file dropped on disk (.ps1)

EventID 11 is file create. The NOT filter removes the harmless __PSScriptPolicyTest files that PowerShell creates by itself.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=11 "Event.EventData.TargetFilename"="*.ps1"
NOT "Event.EventData.TargetFilename"="*__PSScriptPolicyTest*"
| sort + _time
| table _time Event.EventData.Image Event.EventData.TargetFilename Event.EventData.ProcessGuid
```

A4. Firewall rule added

Sysmon does not log firewall rules as their own event. They show up as registry value sets (EventID 13) under FirewallRules. RA4= marks a rule with a remote IPv4 address.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=13 "Event.EventData.TargetObject"="*FirewallRules*"
"Event.EventData.Details"="*RA4=*"
| sort - _time
| table _time Event.EventData.User Event.EventData.Image Event.EventData.Details Event.EventData.TargetObject
```

A5. Process chain (parent and child) by ProcessGuid

Use ProcessGuid, not the process name, to follow one exact process. Replace YOUR-GUID with the ProcessGuid from the event you are tracing.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
("Event.EventData.ProcessGuid"="YOUR-GUID" OR "Event.EventData.ParentProcessGuid"="YOUR-GUID")
| sort + _time
| table _time Event.System.EventID Event.EventData.ParentImage Event.EventData.Image Event.EventData.CommandLine
      Event.EventData.TargetFilename
```

A6. Network connections made by one process

EventID 3 is a network connection. Shows where a suspicious process connected to.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=3 "Event.EventData.ProcessGuid"="YOUR-GUID"
| table _time Event.EventData.SourceIp Event.EventData.DestinationIp Event.EventData.DestinationPort
```

Part B - Password reset / jump box investigation (Windows Security logs)

B1. Check the data exists

Run this first. It confirms the password and logon events are being ingested, and which index and host they come from.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724 OR EventCode=4624)
| stats count by index host EventCode
```

B2. Check the field names

Run this second. If any column comes back empty, that field has a different name in your logs and the later queries need changing to match.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724 OR EventCode=4624)
| head 20
| table _time host EventCode Account_Name Subject_Account_Name Target_Account_Name Logon_Type
      Source_Network_Address Workstation_Name Caller_Computer_Name
```

B3. How many users changed their own password (exact name match)

4723 = an account changed its own password. 4724 = someone reset it. Self change means the account doing it and the account affected are the same.

```
index=* sourcetype=WinEventLog:Security EventCode=4723
| where lower(Subject_Account_Name)=lower(Target_Account_Name)
| stats dc(Target_Account_Name) as users_who_changed_own_password, count as total_changes
```

B4. Same, treating firstname_xxx accounts as one person

Cuts everything after the first underscore, so manish_backup and manish_svc both become manish. Includes 4724 because changing another account of your own shows up as a reset.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724)
| eval subj=lower(Subject_Account_Name), targ=lower(Target_Account_Name)
| eval subj_base=replace(subj,"_.*$",""), targ_base=replace(targ,"_.*$","")
| where subj_base=targ_base
| stats dc(targ_base) as users_who_reset_own_password, count by EventCode
```

B5. Discover which account name pairs exist

Lists every case where the account doing the change differs from the account affected. Use it to see which suffixes really exist in your environment.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724)
| eval subj=lower(Subject_Account_Name), targ=lower(Target_Account_Name)
| where subj!=targ
| stats count by subj targ
| sort - count
```

B6. Entra ID (Azure AD) self-service password reset

Use this if resets happen through the Microsoft self-service portal instead of Windows events.

```
index=* sourcetype="azure:aad:audit"
("Self-service password reset flow activity progress" OR "Reset password (self-service)")
| stats dc(targetResources{}.userPrincipalName) as users, count
```

B7. Okta password reset or update

Use this if the company uses Okta.

```
index=* sourcetype="OktaIM2:log"
(eventType="user.account.reset_password" OR eventType="user.account.update_password")
| stats dc(actor.alternateId) as users, count
```

B8. Raw timeline for one user on one day (abc*)

Shows the real events one after another: RDP logon, explorer.exe start, then the password event. No jump box name needed, the host column shows the machine. Dates are month/day/year, so 10/11/2026 is 11 October.

```
index=* sourcetype=WinEventLog:Security earliest="10/11/2026:00:00:00" latest="10/12/2026:00:00:00"
  (
    ((EventCode=4723 OR EventCode=4724) (Subject_Account_Name="abc*" OR Target_Account_Name="abc*"))
    OR (EventCode=4624 Logon_Type=10 Account_Name="abc*")
    OR (EventCode=4688 New_Process_Name="*\\explorer.exe" Account_Name="abc*")
  )
| sort 0 _time
| table _time host EventCode Subject_Account_Name Target_Account_Name Account_Name New_Process_Name Logon_Type
      Source_Network_Address Caller_Computer_Name _raw
```

B9. Prove the password change came from the jump box (domain controller side)

Password events are logged on the domain controller. This matches each password event to the network logon (type 3) that carried it, using the Logon ID on the same host. The Source_Network_Address or Workstation_Name of that logon should be the jump box.

```
index=* sourcetype=WinEventLog:Security earliest=-24h
  ((EventCode=4723 OR EventCode=4724) OR (EventCode=4624 Logon_Type=3))
| eval sess=if(EventCode=4624, mvindex(Logon_ID,-1), mvindex(Logon_ID,0))
| eventstats values(EventCode) as codes by host sess
| where mvfind(codes,"4624")>=0 AND (mvfind(codes,"4723")>=0 OR mvfind(codes,"4724")>=0)
| sort 0 host sess _time
| table _time host EventCode sess Account_Name Source_Network_Address Workstation_Name Logon_Type _raw
```

B10. MAIN DETECTION: RDP logon, then password change/reset, then target account logs in

Finds the full chain for any user, with no account suffix and no jump box name in the search. Step 1 is an RDP logon by an account that never has its password changed. Step 2 is a password change or reset where both accounts share the first name. Step 3 is the changed account logging in afterwards. The host column on step 1 is the machine that was used. Not tested on your Splunk: try it on a known case first.

```
index=* sourcetype=WinEventLog:Security earliest=-24h
  ((EventCode=4723 OR EventCode=4724) OR (EventCode=4624 (Logon_Type=2 OR Logon_Type=3 OR Logon_Type=10 OR
        Logon_Type=11)))
| eval acct=lower(if(EventCode=4624, mvindex(Account_Name,-1), Target_Account_Name))
| eval subj=lower(Subject_Account_Name)
| where match(acct,"_")
| eval user_base=replace(acct,"_.*$","")
| eval is_pw=if((EventCode=4723 OR EventCode=4724) AND subj!=acct AND replace(subj,"_.*$","")=user_base,1,0)
| eval is_logon=if(EventCode=4624,1,0)
| where is_pw=1 OR is_logon=1
| eval t_pw=if(is_pw=1,_time,null())
| eventstats min(t_pw) as pw_on_acct by acct
| eval is_rdp=if(is_logon=1 AND Logon_Type=10 AND isnull(pw_on_acct),1,0)
| eval stage=case(is_rdp=1,1, is_pw=1,2, is_logon=1 AND isnotnull(pw_on_acct) AND _time>pw_on_acct,3)
| where isnotnull(stage)
| eval t_rdp=if(stage=1,_time,null()), t3=if(stage=3,_time,null())
| eventstats min(t_rdp) as first_rdp min(t_pw) as first_pw max(t3) as last_login by user_base
| where isnotnull(first_rdp) AND isnotnull(first_pw) AND isnotnull(last_login) AND first_rdp<first_pw AND
      last_login>first_pw
| eval step=case(stage=1,"1_RDP_logon", stage=2 AND EventCode=4723,"2_password_CHANGED", stage=2 AND
      EventCode=4724,"2_password_RESET", stage=3,"3_target_account_login")
| sort 0 user_base _time
| table _time host user_base step EventCode acct Subject_Account_Name Target_Account_Name Logon_Type
      Source_Network_Address Workstation_Name Caller_Computer_Name
```

B11. One line per incident (replace the last two lines of B10)

Delete the final sort and table lines of B10 and put these on the end instead. Better for a saved alert, since each incident is one row.

```
| stats earliest(_time) as start, latest(_time) as end, values(acct) as accounts_involved,
      values(Subject_Account_Name) as changed_by, values(host) as hosts, values(Source_Network_Address) as
      source_ips by user_base
| convert ctime(start) ctime(end)
```